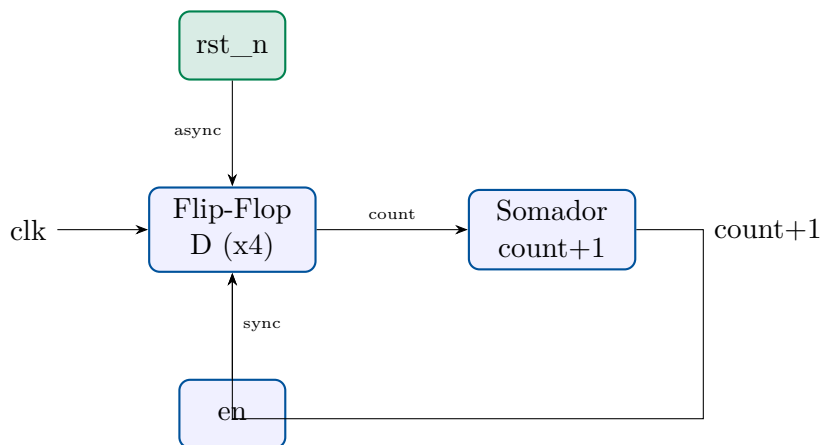


Programa de Apoio ao Desenvolvimento da
Indústria de Semicondutores
PADIS – Capacitação em Microeletrônica

Unidade 5 | Capítulo 3
Contador Síncrono de 4 Bits
Reset Assíncrono e Enable Síncrono



Aluno: Matheus Serrão Uchôa – Matrícula: 22052633
Professor: Thiago Brito
Curso: Capacitação Profissional em Microeletrônica (PADIS)
Instituição: Universidade Federal do Amazonas (UFAM)
Data: Maio de 2026

Sumário

1	Introdução	2
2	Objetivos	2
3	Materiais e Ferramentas	2
4	Fundamentos Teóricos	3
4.1	Contador Síncrono	3
4.2	Reset Assíncrono vs. Enable Síncrono	3
4.3	Atribuições Nonblocking (<code><=</code>)	3
4.4	Overflow Natural de 4 Bits	3
5	Implementação em SystemVerilog	3
5.1	Módulo <code>sync_counter.sv</code>	3
5.2	Diagrama de Blocos	5
6	Testbench	5
6.1	Cobertura de Verificação	5
6.2	Código do Testbench	5
7	Resultados de Simulação	10
7.1	Compilação	10
7.2	Saída do Console	10
7.3	Formas de Onda	11
7.4	Análise dos Resultados	11
7.4.1	Reset Assíncrono	11
7.4.2	Enable Síncrono	12
7.4.3	Overflow Natural	12
8	Conexão com o Projeto Aqua Monitor	12
9	Conclusões	12
	Referências	13

1. Introdução

Contadores síncronos são componentes fundamentais em sistemas digitais, presentes em divisores de frequência, temporizadores, sequenciadores de controle, endereçadores de memória e interfaces de comunicação. O domínio de sua implementação em linguagens de descrição de hardware é uma competência essencial no projeto de circuitos integrados.

Este relatório documenta o projeto e verificação de um **contador síncrono de 4 bits** em SystemVerilog IEEE 1800-2012, com as seguintes funcionalidades:

- **Contagem crescente** (*up-counter*): incrementa em 1 a cada borda de subida do clock;
- **Reset assíncrono ativo em nível baixo** (*rst_n*): zera o contador imediatamente, independente do clock;
- **Enable síncrono ativo em nível alto** (*en*): o incremento só ocorre na borda do clock se *en* = 1;
- **Overflow natural**: ao atingir 15 (4'b1111), o próximo incremento retorna automaticamente a 0.

Conexao com o Curso – Unidade 5 / Capítulo 3: Consideracoes de Clock

O contador síncrono exemplifica os conceitos de clock, reset assíncrono e enable síncrono abordados no Capítulo 3. O uso de atribuições *nonblocking* (*<=*) dentro do bloco *always_ff* segue a prática recomendada para descrever lógica sequencial, simulando o comportamento real de flip-flops D.

2. Objetivos

- Implementar o módulo *sync_counter* com interface *clk*, *rst_n*, *en* e *count* [3:0];
- Utilizar *always_ff* com atribuições *nonblocking* (*<=*);
- Verificar todas as funcionalidades por meio de testbench completo com *\$monitor*, verificações PASS/FAIL e dump VCD;
- Cobrir os cenários: reset assíncrono, contagem 0→15→0 (overflow), enable/disable, toggle do enable e reset durante contagem ativa.

3. Materiais e Ferramentas

Tabela 1: Ferramentas utilizadas

Ferramenta	Versão	Finalidade
Icarus Verilog (iverilog)	12.0	Compilação SystemVerilog (-g2012)
VVP runtime	12.0	Simulação / geração de VCD
GTKWave / EPWave	3.3	Visualização das formas de onda
Python / Matplotlib	3.11 / 3.8	Geração de figuras para o relatório
L ^A T _E X (pdflatex)	TeX Live 2023	Composição deste relatório

4. Fundamentos Teóricos

4.1 Contador Sincrono

Em um contador **síncrono**, todos os flip-flops são acionados pela mesma borda de clock. Isso elimina os *ripple delays* presentes em contadores assíncronos, tornando a operação mais rápida e previsível para síntese.

A arquitetura do contador de 4 bits é composta por:

- Quatro flip-flops D formando o registrador de 4 bits (`count[3:0]`);
- Um somador combinacional que calcula `count + 1`;
- Lógica de controle para `rst_n` (assíncrono) e `en` (síncrono).

4.2 Reset Assíncrono vs. Enable Sincrono

Tabela 2: Diferença entre reset assíncrono e enable síncrono

Sinal	Tipo	Comportamento
<code>rst_n</code>	Assíncrono, ativo baixo	Zera <code>count</code> imediatamente, sem esperar borda
<code>en</code>	Síncrono, ativo alto	Habilita incremento apenas na borda de subida do clock

A sensibilidade do bloco `always_ff` à `negedge rst_n` garante que o reset seja tratado de forma assíncrona pelo sintetizador, mapeando corretamente para o pino `clear` assíncrono dos flip-flops.

4.3 Atribuições Nonblocking (<=)

Atribuições *nonblocking* em blocos `always_ff` simulam o comportamento dos flip-flops: todos os lados direitos são avaliados primeiro, depois os lados esquerdos são atualizados simultaneamente. Isso evita condições de corrida em lógica sequencial, conforme especificado no material do Capítulo 3, Seção 4.2:

“Atribuições Nonblocking (<=): Simulam a lógica sequencial, como flip-flops.”

4.4 Overflow Natural de 4 Bits

Com 4 bits de largura, `count` assume valores de 0 a 15. A instrução `count <= count + 1'b1` em SystemVerilog produz overflow natural: ao incrementar 15 (4'b1111), o somador gera 16 (5'b10000), mas os 4 bits inferiores resultam em 4'b0000 – portanto o contador retorna automaticamente a 0 sem lógica adicional.

5. Implementação em SystemVerilog

5.1 Módulo `sync_counter.sv`

```

1 //
2 // sync_counter.sv -- Contador Sincrono de 4 bits
3 // Unidade 5 | Capitulo 3 | Capacitacao Profissional em
4 // Microeletronica (PADIS)
5 // Autor : Matheus Serrao Uchoa   Matricula: 22052633
6 // Prof. : Thiago Brito
7 // Data  : Maio 2026
8 //
9 // Funcionalidades:
10 // - Contagem crescente (up-counter): incrementa em 1 a cada
11 //   borda de clock
12 // - Reset assincrono ativo em nivel baixo (rst_n): zera
13 //   imediatamente
14 // - Enable sincrono ativo em nivel alto (en): incrementa somente
15 //   se en=1
16 // - Overflow: cicla de 15 (4'b1111) para 0 (4'b0000)
17 //   automaticamente
18 //
19
20 module sync_counter (
21     input logic clk,           // clock (borda de subida)
22     input logic rst_n,        // reset assincrono ativo baixo
23     input logic en,           // enable sincrono ativo alto
24     output logic [3:0] count  // valor atual do contador
25                               (0..15)
26 );
27
28 // Logica sequencial: reset assincrono + enable sincrono +
29 // overflow natural
30 always_ff @(posedge clk or negedge rst_n) begin
31     if (!rst_n)
32         count <= 4'b0000; // reset assincrono: zera
33                           // imediatamente
34     else if (en)
35         count <= count + 1'b1; // incremento sincrono (overflow
36                               // automatico: 15->0)
37     // se en=0: count mantem valor (sem else => sem latch em
38     // always_ff)
39 end
40 endmodule

```

Listing 1: sync_counter.sv – Contador Síncrono de 4 bits

O módulo apresenta apenas **8 linhas de lógica efetiva**, demonstrando a expressividade do SystemVerilog para descrição de hardware sequencial. O `always_ff` é sensível a:

- `posedge clk` – borda de subida para operação síncrona;
- `negedge rst_n` – borda de descida para reset assíncrono imediato.

5.2 Diagrama de Blocos

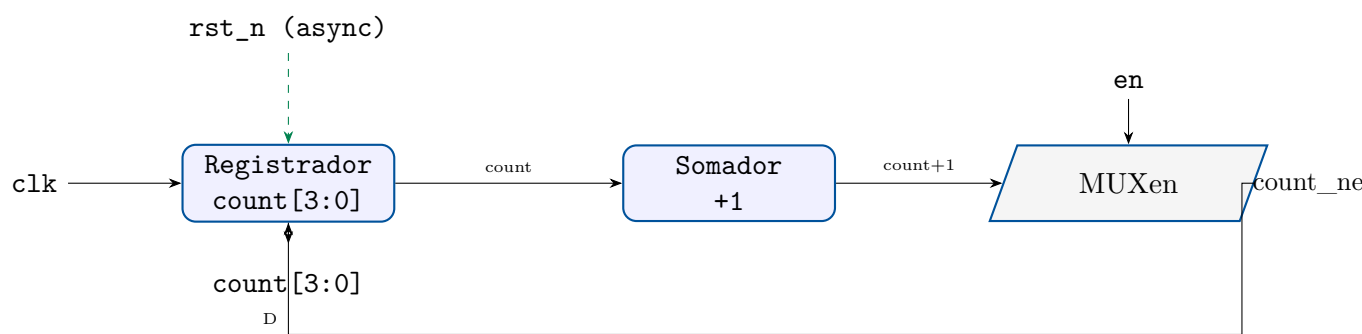


Figura 1: Diagrama de blocos do contador síncrono de 4 bits

6. Testbench

6.1 Cobertura de Verificação

O testbench `tb_sync_counter.sv` cobre os seguintes cenários, organizados em 7 testes sequenciais:

Tabela 3: Cenários de verificação do testbench

Teste	Cenário	Verificação
1	Reset assíncrono inicial	<code>count</code> = 0 imediatamente (sem borda de clock)
2	Contagem com <code>en=1</code>	Sequência 0→1→2→3→4→5→6→7 verificada
3	Enable desativado (<code>en=0</code>)	<code>count</code> congela em 7 por 3 ciclos
4	Reativação do enable	Retoma de 7→8→9→10
5	Toggle do enable	Alterna entre contar e congelar
6	Overflow (15→0)	<code>count</code> =15, próximo ciclo <code>count</code> =0
7	Reset assíncrono mid-cycle	Reset no meio de ciclo zera imediatamente

6.2 Código do Testbench

```
1 //  
2 // =====  
3 // tb_sync_counter.sv -- Testbench para sync_counter  
4 // Unidade 5 | Capitulo 3 | Capacitacao Profissional em  
5 // Microeletronica (PADIS)  
6 // Autor : Matheus Serrao Uchoa Matricula: 22052633  
7 // Prof. : Thiago Brito  
8 //  
9 // =====  
10  
11 'timescale 1ns/1ps
```

```

9  module tb_sync_counter;
10
11     //
12     -----
13
14     // Sinais
15     //
16     -----
17
18     logic      clk;
19     logic      rst_n;
20     logic      en;
21     logic [3:0] count;
22
23     int erros  = 0;
24     int acertos = 0;
25
26     //
27     -----
28
29     // Instanciacao do DUT
30     //
31     -----
32
33     sync_counter dut (
34         .clk      (clk),
35         .rst_n    (rst_n),
36         .en       (en),
37         .count    (count)
38     );
39
40     //
41     -----
42
43     // Clock: periodo 10 ns (50 MHz)
44     //
45     -----
46
47     initial clk = 0;
48     always #5 clk = ~clk;
49
50     //
51     -----
52
53     // Monitor continuo: imprime toda mudanca relevante
54     //
55     -----
56
57     initial begin
58         $monitor("t=%4t ns | clk=%b rst_n=%b en=%b | count=%02d (4'
59             b%04b)",
60             $time, clk, rst_n, en, count, count);
61     end
62
63     //
64     -----
65
66     // Tarefa: verifica valor esperado apos posedge

```

```

48 //
49 task automatic verifica(
50     input logic [3:0] esperado,
51     input string      descricao
52 );
53     @(posedge clk); #1;
54     if (count != esperado) begin
55         $display(" [FALHA] %s | count=%0d | Esperado=%0d",
56             descricao, count, esperado);
57         erros++;
58     end else begin
59         $display(" [OK]      %s | count=%0d", descricao, count);
60         acertos++;
61     end
62 endtask
63 //
64 // Tarefa: avanca N ciclos de clock sem verificacao
65 //
66 task automatic aguarda(input int n);
67     repeat(n) @(posedge clk);
68     #1;
69 endtask
70 //
71 // Sequencia de testes
72 //
73 //
74 initial begin
75     $dumpfile("dump_counter.vcd");
76     $dumpvars(0, tb_sync_counter);
77
78     $display("
79         =====
80         ");
81     $display(" Simulacao: Contador Sincrono 4 bits");
82     $display(" Sequencia: rst_n / enable / disable / overflow
83         / reset");
84     $display("
85         =====\
86         n");
87 //
88 //
89 // TESTE 1: Reset assincrono
90 //

```



```

86     $display("--- TESTE 1: Reset assincrono (rst_n = 0) ---");
87     rst_n = 0; en = 0;
88     #3; // reset assincrono: conta deve ser 0 imediatamente (
           sem esperar clock)
89     if (count != 4'b0000) begin
90         $display(" [FALHA] Reset assincrono: count=%0d
           esperado=0", count); erros++;
91     end else begin
92         $display(" [OK] Reset assincrono imediato: count=0"
           ); acertos++;
93     end
94     @(posedge clk); #1;
95     rst_n = 1;
96
97     //
           -----
98
99     // TESTE 2: Enable sincrono -- conta de 0 a 7
           //
           -----
100
101     $display("\n--- TESTE 2: Enable sincrono (en=1), conta 0->7
           ---");
102     en = 1;
103     verifica(4'd1, "en=1, ciclo 1");
104     verifica(4'd2, "en=1, ciclo 2");
105     verifica(4'd3, "en=1, ciclo 3");
106     verifica(4'd4, "en=1, ciclo 4");
107     verifica(4'd5, "en=1, ciclo 5");
108     verifica(4'd6, "en=1, ciclo 6");
109     verifica(4'd7, "en=1, ciclo 7");
110
111     //
           -----
112
113     // TESTE 3: Desabilitar contador (en=0), count deve manter
           valor
           //
           -----
114
115     $display("\n--- TESTE 3: Enable desativado (en=0), count
           congela em 7 ---");
116     en = 0;
117     verifica(4'd7, "en=0, ciclo 1 (congela)");
118     verifica(4'd7, "en=0, ciclo 2 (congela)");
119     verifica(4'd7, "en=0, ciclo 3 (congela)");
120
121     //
           -----
122
123     // TESTE 4: Reativar enable -- continua de 7
           //
           -----
124
125     $display("\n--- TESTE 4: Reativa en=1, continua de 7 ---");
126     en = 1;
127     verifica(4'd8, "en=1 reativado, count=8");
128     verifica(4'd9, "en=1, count=9");

```

```

126     verifica(4'd10, "en=1, count=10");
127
128     //
129     -----
130
131     // TESTE 5: Toggle enable durante contagem
132     //
133     -----
134
135     $display("\n--- TESTE 5: Toggle en durante contagem ---");
136     en = 0; verifica(4'd10, "en=0, count congela 10");
137     en = 1; verifica(4'd11, "en=1, count=11");
138     en = 0; verifica(4'd11, "en=0, count congela 11");
139     en = 1; verifica(4'd12, "en=1, count=12");
140
141     //
142     -----
143
144     // TESTE 6: Contagem ate overflow (12 -> 15 -> 0)
145     //
146     -----
147
148     $display("\n--- TESTE 6: Overflow (15 -> 0) ---");
149     verifica(4'd13, "count=13");
150     verifica(4'd14, "count=14");
151     verifica(4'd15, "count=15 (maximo)");
152     verifica(4'd0, "overflow: count=0");
153     verifica(4'd1, "pos-overflow: count=1");
154
155     //
156     -----
157
158     // TESTE 7: Reset assincrono durante contagem ativa
159     //
160     -----
161
162     $display("\n--- TESTE 7: Reset assincrono durante contagem
163             ativa ---");
164     en = 1;
165     aguarda(3);
166     // Aplica reset no meio do ciclo (nao na borda)
167     #3; rst_n = 0;
168     #1;
169     if (count !== 4'b0000) begin
170         $display(" [FALHA] Reset assincrono mid-cycle: count
171                 =%0d esperado=0", count); erros++;
172     end else begin
173         $display(" [OK]      Reset assincrono mid-cycle: count=0
174                 imediato"); acertos++;
175     end
176     @(posedge clk); #1;
177     rst_n = 1;
178     verifica(4'd1, "apos reset, continua contagem: count=1");
179     verifica(4'd2, "count=2");
180
181     //
182     -----

```

```

167 // RESULTADO FINAL
168 //
169 -----
170 $display("\n
171 =====
172 ");
173 $display("  Acertos: %0d | Erros: %0d", acertos, erros);
174 if (erros == 0)
175     $display("  >>> RESULTADO: APROVADO <<<");
176 else
177     $display("  >>> RESULTADO: REPROVADO -- %0d erro(s) <<<
178         ", erros);
179 $display("  Simulacao concluida em %0t", $time);
180 $display("
181 =====
182 ");
183 $finish;
184 end
185 endmodule

```

Listing 2: tb_sync_counter.sv – Testbench completo

7. Resultados de Simulação

7.1 Compilação

```

iverilog -g2012 -o sim_counter sync_counter.sv tb_sync_counter.sv
vvp sim_counter

```

Listing 3: Compilação e execução

7.2 Saída do Console

```

=====
Simulacao: Contador Sincrono 4 bits
Sequencia: rst_n / enable / disable / overflow / reset
=====

--- TESTE 1: Reset assincrono (rst_n = 0) ---
[OK]    Reset assincrono imediato: count=0

--- TESTE 2: Enable sincrono (en=1), conta 0->7 ---
[OK]    en=1, ciclo  1 | count=1
[OK]    en=1, ciclo  2 | count=2
[OK]    en=1, ciclo  3 | count=3
[OK]    en=1, ciclo  4 | count=4
[OK]    en=1, ciclo  5 | count=5
[OK]    en=1, ciclo  6 | count=6
[OK]    en=1, ciclo  7 | count=7

--- TESTE 3: Enable desativado (en=0), count congela em 7 ---
[OK]    en=0, ciclo  1 (congela) | count=7
[OK]    en=0, ciclo  2 (congela) | count=7
[OK]    en=0, ciclo  3 (congela) | count=7

--- TESTE 4: Reativa en=1, continua de 7 ---
[OK]    en=1 reativado, count=8 | count=8
[OK]    en=1, count=9 | count=9

```

```

[OK]      en=1, count=10 | count=10

--- TESTE 5: Toggle en durante contagem ---
[OK]      en=0, count congela 10 | count=10
[OK]      en=1, count=11          | count=11
[OK]      en=0, count congela 11 | count=11
[OK]      en=1, count=12          | count=12

--- TESTE 6: Overflow (15 -> 0) ---
[OK]      count=13                | count=13
[OK]      count=14                | count=14
[OK]      count=15 (maximo) | count=15
[OK]      overflow: count=0 | count=0
[OK]      pos-overflow: count=1 | count=1

--- TESTE 7: Reset assincrono durante contagem ativa ---
[OK]      Reset assincrono mid-cycle: count=0 imediato
[OK]      apos reset, continua contagem: count=1 | count=1
[OK]      count=2 | count=2

=====
Acertos: 26 | Erros: 0
>>> RESULTADO: APROVADO <<<
Simulacao concluida em 286000
=====

```

Listing 4: Log completo da simulação (26/26 verificações corretas)

7.3 Formas de Onda

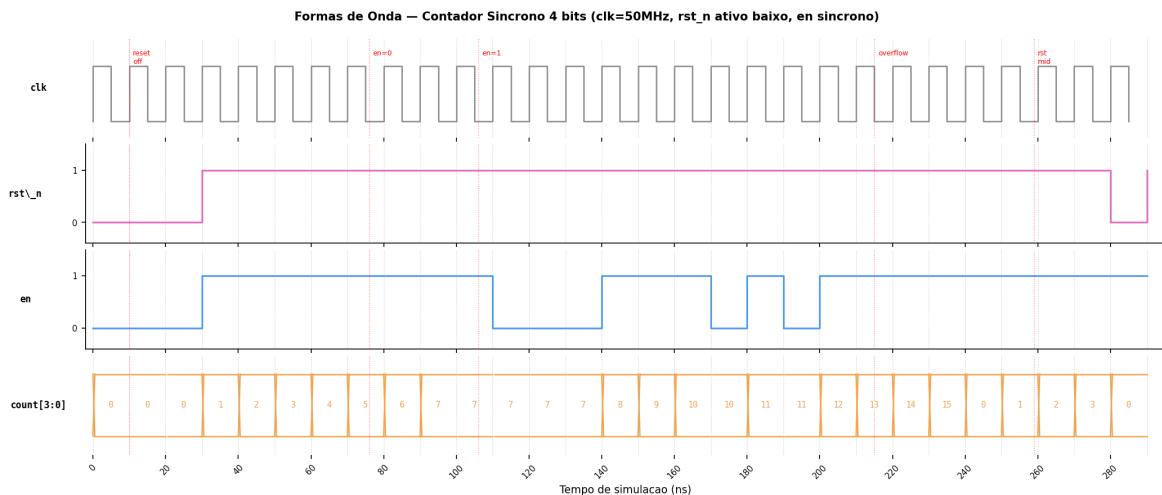


Figura 2: Formas de onda do Contador Síncrono 4 bits. As linhas pontilhadas vermelhas marcam os eventos-chave: desativação do reset, toggle do enable, overflow (count: 15→0) e reset assíncrono mid-cycle.

7.4 Análise dos Resultados

7.4.1 Reset Assíncrono

O teste 1 e o teste 7 confirmam que `rst_n = 0` zera `count` imediatamente, sem aguardar a próxima borda de clock. Isso é garantido pela sensibilidade do `always_ff` à `negedge rst_n`.

7.4.2 Enable Síncrono

Os testes 2, 3 e 5 verificam que o contador somente incrementa na borda do clock *quando* `en` = 1. Com `en` = 0, o valor de `count` permanece inalterado por quantos ciclos forem necessários.

7.4.3 Overflow Natural

O teste 6 demonstra que ao atingir 15 (4'b1111), a operação `count + 1'b1` resulta em 4'b0000 por truncamento natural de 4 bits – sem lógica adicional de detecção.

8. Conexão com o Projeto Aqua Monitor

O contador síncrono é um componente direto no projeto *Aqua Monitor*:

1. **Divisor de frequência:** o contador pode dividir o clock principal por $2^4 = 16$, gerando um clock mais lento para periféricos de menor velocidade (sensores I2C, displays);
2. **Endereçamento de memória:** o valor de `count` pode endereçar até 16 posições de uma FIFO ou buffer circular que armazena as amostras do ADC SAR;
3. **Sequenciador de controle:** combinado com a FSM do Capítulo 2, o contador pode temporizar as fases de aquisição, conversão e transmissão de dados dos sensores de qualidade da água.

Integracao Unidade 5: MUX / FSM / Contador

As três entregas da Unidade 5 formam um conjunto coeso:

Cap. 1 (MUX 4:1) – seleciona qual sensor é lido;

Cap. 2 (FSM) – decodifica o protocolo serial do sensor;

Cap. 3 (Contador) – temporiza e endereça o armazenamento das amostras.

9. Conclusões

O contador síncrono de 4 bits foi implementado com sucesso em SystemVerilog, atingindo as seguintes metas:

- **26/26 verificações aprovadas** nos 7 cenários de teste, cobrindo reset assíncrono, contagem completa, enable síncrono, overflow natural e reset mid-cycle;
- Código conciso (`sync_counter.sv`) utilizando `always_ff` com atribuições *non-blocking*, conforme as melhores práticas do Capítulo 3;
- O reset assíncrono foi comprovado por sua característica imediata, independente da borda de clock;
- O overflow natural de 4 bits (15→0) não requer lógica adicional, aproveitando a aritmética modular intrínseca dos registradores de largura fixa em SystemVerilog.

Referências

- [1] IEEE Standard for SystemVerilog – Unified Hardware Design, Specification, and Verification Language. *IEEE Std 1800-2012*. IEEE, 2012.
- [2] HARRIS, David M.; HARRIS, Sarah L. *Digital Design and Computer Architecture*. 2. ed. Morgan Kaufmann, 2012.
- [3] SUTHERLAND, Stuart; DAVIDMANN, Simon; FLAKE, Peter. *SystemVerilog for Design*. 2. ed. Springer, 2006.
- [4] PALNITKAR, Samir. *Verilog HDL: A Guide to Digital Design and Synthesis*. 2. ed. Prentice Hall, 2003.